SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided

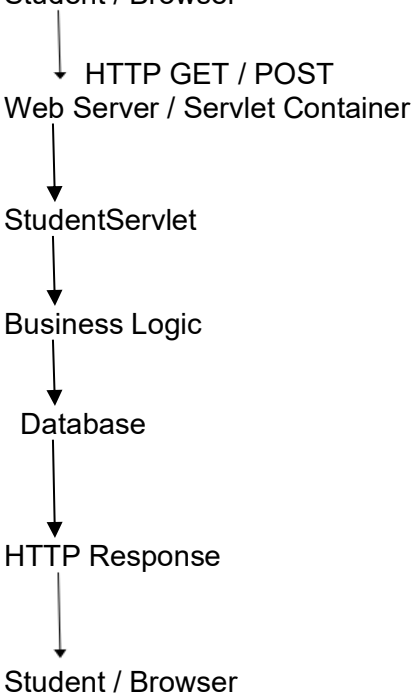
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1.Online Student Registration System Using Java Servlet

Scenario

A college wants to develop an online student registration system. Students enter their details through a web form, and the data is processed using a Java Servlet running on Apache Tomcat.

Servlet Architecture



Required HTML Form Code

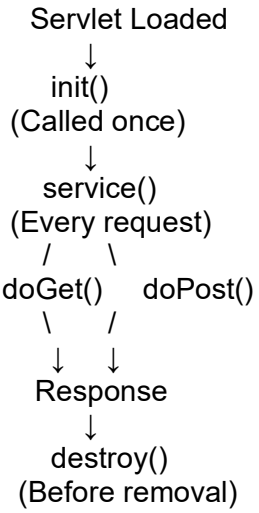
```
<form action="StudentServlet" method="post">

    Name:
    <input type="text" name="name">

    Email:
    <input type="email" name="email">

    Course:
    <input type="text" name="course">

    <input type="submit" value="Register">
</form>
```

Required Servlet Code

```
public class StudentServlet extends HttpServlet {
```

```
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");

        response.getWriter().println(
            "Student: " + name + "<br>Email: " + email);
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        response.getWriter().println(
            "<h2>Registration Successful</h2>" +
            "Name: " + name + "<br>" +
            "Email: " + email + "<br>" +
            "Course: " + course);
    }
}
```

Servlet Life Cycle



- **init()** – Initializes the servlet.
- **service()** – Handles incoming requests.
- **doGet() / doPost()** – Process the corresponding HTTP method.
- **destroy()** – Releases resources before the servlet is removed.

GET vs POST

GET	POST
Data appears in URL	Data sent in request body
Limited data	Supports larger data
Less suitable for sensitive information	Preferred for form submission
Used mainly for retrieving data	Used mainly for submitting data

Conclusion: POST is more appropriate for student registration because personal information is not exposed in the URL. For actual security, **HTTPS must also be used**.

Expected Output

Registration Successful

Name : Alice
Email : alice@gmail.com
Course : BCA

2.Login and Session Management

Scenario

A college website allows students to log in and access multiple pages such as Profile, Courses, and Results without logging in again on every page.

Session Management Flow

Login Page
↓
LoginServlet
↓
Validate Username & Password
↓
Create HttpSession
↓
Store Username
↓
Session ID → Browser Cookie
↓
Other Pages
↓
Server Retrieves Session
↓
Student Remains Logged In

Required Login Code

```
String username = request.getParameter("username");
String password = request.getParameter("password");

if (username.equals("admin") &&
    password.equals("123")) {

    HttpSession session = request.getSession();

    session.setAttribute("username", username);

    response.sendRedirect("HomeServlet");
}
```

Required Session Retrieval Code

```
HttpSession session =
        request.getSession(false);

if (session != null) {

    String username =
        (String) session.getAttribute("username");

    response.getWriter().println(
        "Welcome " + username);

} else {

    response.getWriter().println(
        "Please Login");
}
```

Cookie Code

```
Cookie cookie =
    new Cookie("username", username);

cookie.setMaxAge(3600);

response.addCookie(cookie);
```

Session Tracking

HTTP is stateless, so the server needs a mechanism to identify the same user across different requests.

```
Student Login
    ↓
HttpSession Created
    ↓
Session ID Generated
    ↓
Session ID Stored in Cookie
```

↓
Next Request Contains Session ID
↓
Server Finds Session
↓
Student Remains Logged In

HttpSession vs Cookies

HttpSession	Cookies
Data stored on server	Data stored in browser
Suitable for login sessions	Suitable for small preferences/identifiers
Can store objects	Stores text values
Session expires based on timeout/logout	Expiration can be configured

Conclusion: HttpSession is preferred for maintaining authenticated student sessions. Cookies can carry the session identifier and store non-sensitive client-side information.

Expected Output

Login Page

Username: admin
Password: ****

↓

Welcome admin

Home | Profile | Courses | Results | Logout

After logout/session expiration:

Please Login

3.JDBC-Based E-Commerce Product System

Scenario

An e-commerce company wants to store product information such as product ID, name, and price in a MySQL database and retrieve it through a Java application.

JDBC Architecture

Java Application
↓
JDBC API
↓
DriverManager
↓
JDBC Driver
↓
MySQL Database

Product Table

```
CREATE TABLE product (
   id INT PRIMARY KEY,
   name VARCHAR(100),
   price DOUBLE
);
```

JDBC Connectivity Steps

Load Driver
↓
Create Connection
↓
Create Statement
↓
Execute SQL
↓
Process ResultSet
↓
Close Resources

Required Connection Code

```
Connection con = DriverManager.getConnection(
   "jdbc:mysql://localhost:3306/shop",
   "root",
   "password");
```

Required Insert Code

```
String sql =
   "INSERT INTO product VALUES (?, ?, ?)";

PreparedStatement ps =
   con.prepareStatement(sql);

ps.setInt(1, 101);
ps.setString(2, "Laptop");
ps.setDouble(3, 65000);

ps.executeUpdate();
```

Required Retrieval Code

```
Statement st = con.createStatement();

ResultSet rs =
   st.executeQuery("SELECT * FROM product");

while (rs.next()) {

   System.out.println(
      rs.getInt("id") + " " +
      rs.getString("name") + " " +
      rs.getDouble("price"));
}
```

Exception Handling

```
try {
   // JDBC operations

} catch (SQLException e) {

   System.out.println(
      "Database Error: " + e.getMessage());
}
```

Why PreparedStatement?

PreparedStatement is used for inserting product data because it:

- Prevents SQL injection.
- Allows parameters using ?.
- Makes SQL queries safer and easier to manage.

Expected Output

After inserting a product:

```
1 Record Inserted
```

After retrieving products:

```
101 Laptop 65000.0
102 Mobile 25000.0
103 Headphone 1500.0
```

Scenario-Based JDBC Flow

Administrator
↓
Java E-Commerce Application
↓
JDBC
↓
MySQL Database
↓
Product Stored

Customer
↓
Product Search
↓
JDBC SELECT Query
↓
MySQL Database
↓
ResultSet
↓
Products Displayed

Conclusion: JDBC provides the connection between the Java e-commerce application and the database, allowing products to be inserted and retrieved using SQL queries.